

2026 SISBID Clustering Demo

Genevera I. Allen & Yufeng Liu

Packages and helper functions

```
library(ggplot2)

load_nci60 <- function() {
  if (requireNamespace("ISLR", quietly = TRUE)) {
    data("NCI60", package = "ISLR", envir = environment())
    return(NCI60)
  }
  if (requireNamespace("ISLR2", quietly = TRUE)) {
    data("NCI60", package = "ISLR2", envir = environment())
    return(NCI60)
  }
  stop("Please install either the ISLR or ISLR2 package to use the NCI60 data.")
}

adjusted_rand_index <- function(x, y) {
  tab <- table(x, y)
  choose2 <- function(z) z * (z - 1) / 2
  n <- sum(tab)
  index <- sum(choose2(tab))
  row_index <- sum(choose2(rowSums(tab)))
  col_index <- sum(choose2(colSums(tab)))
  expected <- row_index * col_index / choose2(n)
  max_index <- 0.5 * (row_index + col_index)
  denom <- max_index - expected
  if (denom == 0) return(NA_real_)
  (index - expected) / denom
}

plot_cluster_solution <- function(data, xvar = "x1", yvar = "x2", cluster_var,
                                  center_data = NULL, title = NULL) {
  p <- ggplot(data, aes(x = .data[[xvar]], y = .data[[yvar]], color = .data[[cluster_var]])) +
    geom_point(size = 2, alpha = 0.85) +
    labs(title = title, color = cluster_var) +
    theme_minimal()

  if (!is.null(center_data)) {
    p <- p +
      geom_point(
        data = center_data,
        aes(x = .data[[xvar]], y = .data[[yvar]]),
        inherit.aes = FALSE,
        size = 5,

```

```

        shape = 4,
        stroke = 1.5
      )
    }
  }
  p
}

plot_kmeans_diagnostics <- function(X, k_grid = 1:10, nstart = 25, seed = 2107) {
  set.seed(seed)
  wss <- sapply(k_grid, function(k) {
    kmeans(X, centers = k, nstart = nstart)$tot.withinss
  })

  diag_data <- data.frame(K = k_grid, total_withinss = wss)

  ggplot(diag_data, aes(x = K, y = total_withinss)) +
    geom_line() +
    geom_point(size = 2) +
    scale_x_continuous(breaks = k_grid) +
    labs(
      title = "K-means elbow diagnostic",
      x = "Number of clusters K",
      y = "Total within-cluster sum of squares"
    ) +
    theme_minimal()
}

pca_scores_for_plot <- function(X, groups, labels = NULL, center = FALSE, scale. = FALSE) {
  pca <- prcomp(X, center = center, scale. = scale.)
  var_explained <- 100 * pca$sdev^2 / sum(pca$sdev^2)

  out <- data.frame(
    PC1 = pca$x[, 1],
    PC2 = pca$x[, 2],
    group = factor(groups),
    label = labels
  )

  attr(out, "pca") <- pca
  attr(out, "var_explained") <- var_explained
  out
}

plot_pca_clusters <- function(score_data, title = NULL, show_text = TRUE) {
  var_explained <- attr(score_data, "var_explained")

  p <- ggplot(score_data, aes(x = PC1, y = PC2, color = group)) +
    geom_point(size = 2.5, alpha = 0.85) +
    labs(
      title = title,
      x = paste0("PC1 (", round(var_explained[1], 1), "% variance)"),
      y = paste0("PC2 (", round(var_explained[2], 1), "% variance)"),
      color = "Cluster"
    )
}

```

```

) +
  theme_minimal()

if (show_text && !all(is.na(score_data$label))) {
  p <- p + geom_text(aes(label = label), size = 3, check_overlap = TRUE)
}
p
}

spectral_clustering <- function(X, K, sigma = NULL, n_neighbors = 7,
                                nstart = 25, seed = 2107) {
  # Teaching implementation of normalized spectral clustering.
  # This is written for small-to-moderate data sets where a full distance
  # matrix is acceptable.
  D <- as.matrix(dist(X))

  if (is.null(sigma)) {
    sigma <- median(D[D > 0])
  }

  W <- exp(-D^2 / (2 * sigma^2))
  diag(W) <- 0

  if (!is.null(n_neighbors)) {
    nn <- apply(D, 1, function(z) rank(z, ties.method = "first") <= n_neighbors + 1)
    W <- W * (nn | t(nn))
  }

  degree <- rowSums(W)
  D_inv_sqrt <- diag(1 / sqrt(pmax(degree, .Machine$double.eps)))
  L_sym <- diag(nrow(W)) - D_inv_sqrt %*% W %*% D_inv_sqrt

  eig <- eigen(L_sym, symmetric = TRUE)
  ord <- order(eig$values)
  U <- eig$vectors[, ord[1:K], drop = FALSE]
  U <- U / sqrt(rowSums(U^2))

  set.seed(seed)
  km <- kmeans(U, centers = K, nstart = nstart)

  list(cluster = km$cluster, embedding = U, eigenvalues = eig$values[ord], sigma = sigma)
}

```

K-means clustering

Data set 1: simulated data

We first simulate two-dimensional data from a mixture of three normal distributions. Since we know the generating labels, this toy example lets us compare the true groups with the clusters found by k-means.

```
set.seed(2107)
```

```
n <- 300
```

```

mu1 <- c(3, 3)
mu2 <- c(7, 4)
mu3 <- c(5.5, 5.5)
Sigma <- matrix(c(1, 0.5, 0.5, 1), 2, 2)

x1 <- t(matrix(mu1, 2, n / 3)) + matrix(rnorm(n * 2 / 3), n / 3, 2)
xx <- matrix(rnorm(n * 2 / 3), n / 3, 2)
x2 <- t(matrix(mu2, 2, n / 3)) + xx %*% chol(Sigma)
xx <- matrix(rnorm(n * 2 / 3), n / 3, 2)
x3 <- t(matrix(mu3, 2, n / 3)) + xx %*% chol(Sigma)

X_sim <- rbind(x1, x2, x3)
true_label <- factor(rep(1:3, each = n / 3))

sim_data <- data.frame(
  x1 = X_sim[, 1],
  x2 = X_sim[, 2],
  true_label = true_label
)

```

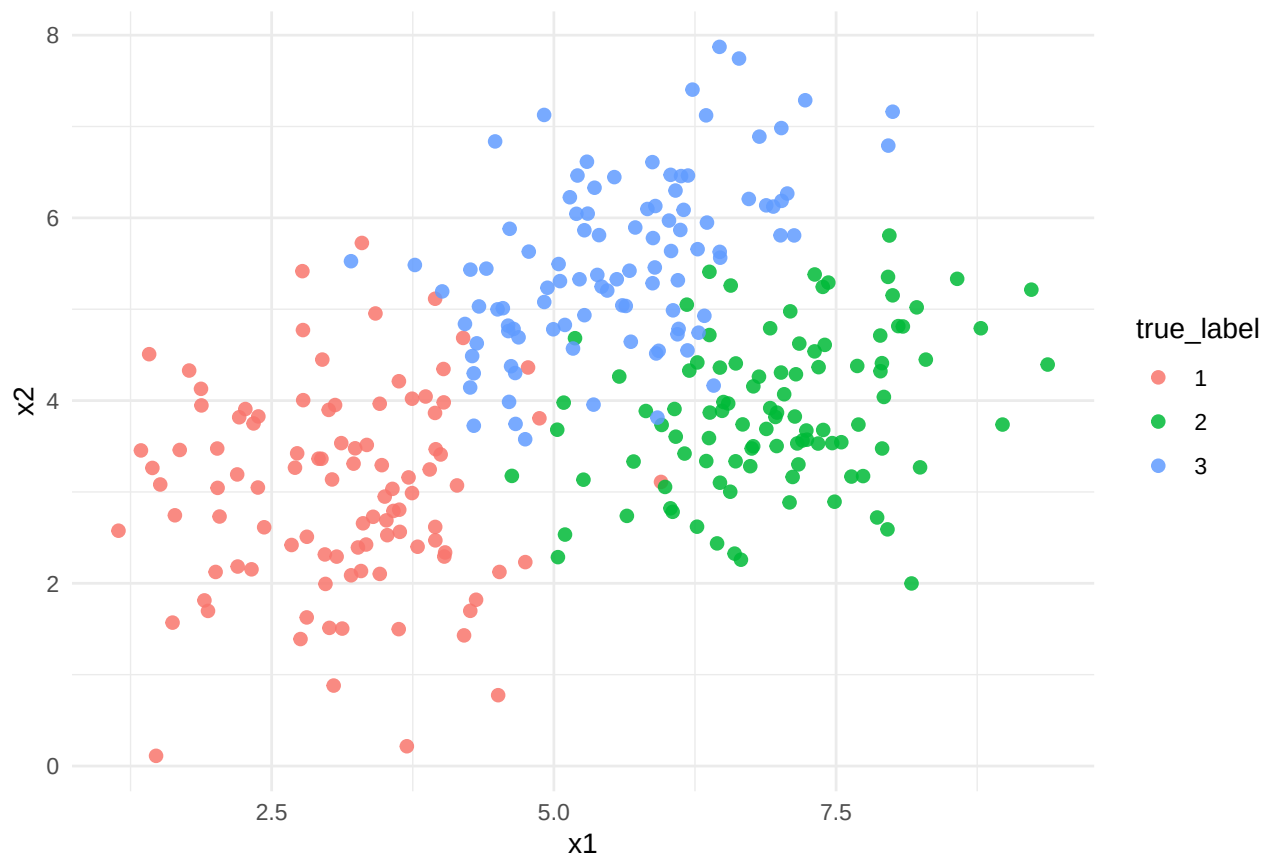
Plot the true labels

```

plot_cluster_solution(
  sim_data,
  cluster_var = "true_label",
  title = "Simulated data: true generating labels"
)

```

Simulated data: true generating labels



Fit k-means with the correct value of K

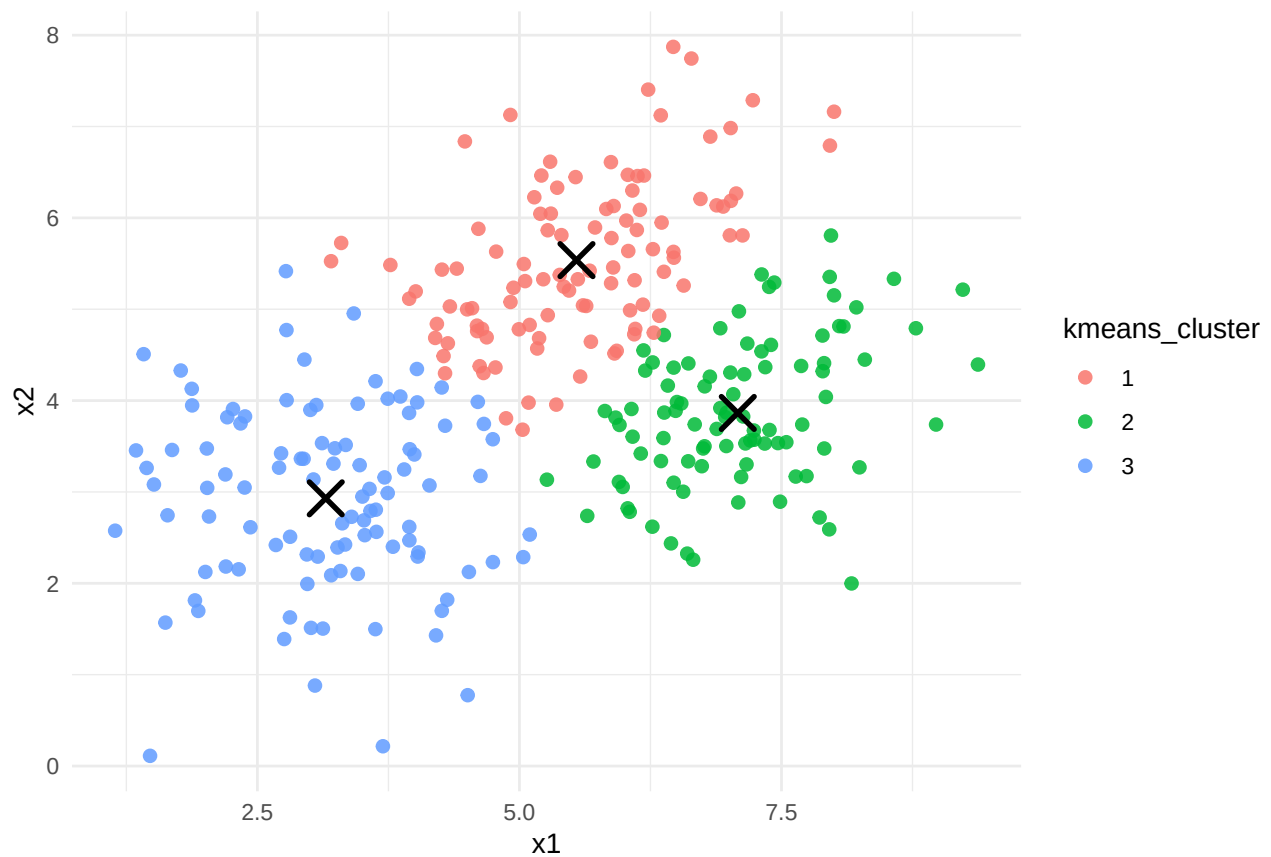
Use `nstart > 1` so that R tries several random initializations and keeps the solution with the smallest within-cluster sum of squares.

```
set.seed(2107)
K <- 3
km_sim <- kmeans(X_sim, centers = K, nstart = 50)

sim_data$kmeans_cluster <- factor(km_sim$cluster)
sim_centers <- data.frame(
  x1 = km_sim$centers[, 1],
  x2 = km_sim$centers[, 2]
)

plot_cluster_solution(
  sim_data,
  cluster_var = "kmeans_cluster",
  center_data = sim_centers,
  title = "K-means solution with K = 3"
)
```

K-means solution with K = 3



```
table(True = sim_data$true_label, Kmeans = sim_data$kmeans_cluster)
```

```
##      Kmeans
## True  1  2  3
##    1  5  1 94
##    2  7 90  3
##    3 92  3  5
```

```
adjusted_rand_index(sim_data$true_label, sim_data$kmeans_cluster)
```

```
## [1] 0.7730895
```

Visualizing the k-means algorithm

The following code manually steps through several one-iteration k-means updates. It is meant to show the mechanics: assign points to the closest center, then update centers.

```
set.seed(2026)
k <- 3
n_sim <- nrow(X_sim)
centers <- X_sim[sample(seq_len(n_sim), k), , drop = FALSE]

iteration_data <- list()
center_data <- list()

for (iter in 0:4) {
  if (iter == 0) {
```

```

    cluster_id <- rep("initial", n_sim)
  } else {
    km_one <- kmeans(X_sim, centers = centers, iter.max = 1, nstart = 1, algorithm = "MacQueen")
    cluster_id <- km_one$cluster
    centers <- km_one$centers
  }

iteration_data[[iter + 1]] <- data.frame(
  x1 = X_sim[, 1],
  x2 = X_sim[, 2],
  cluster = factor(cluster_id),
  iteration = paste("Iteration", iter)
)

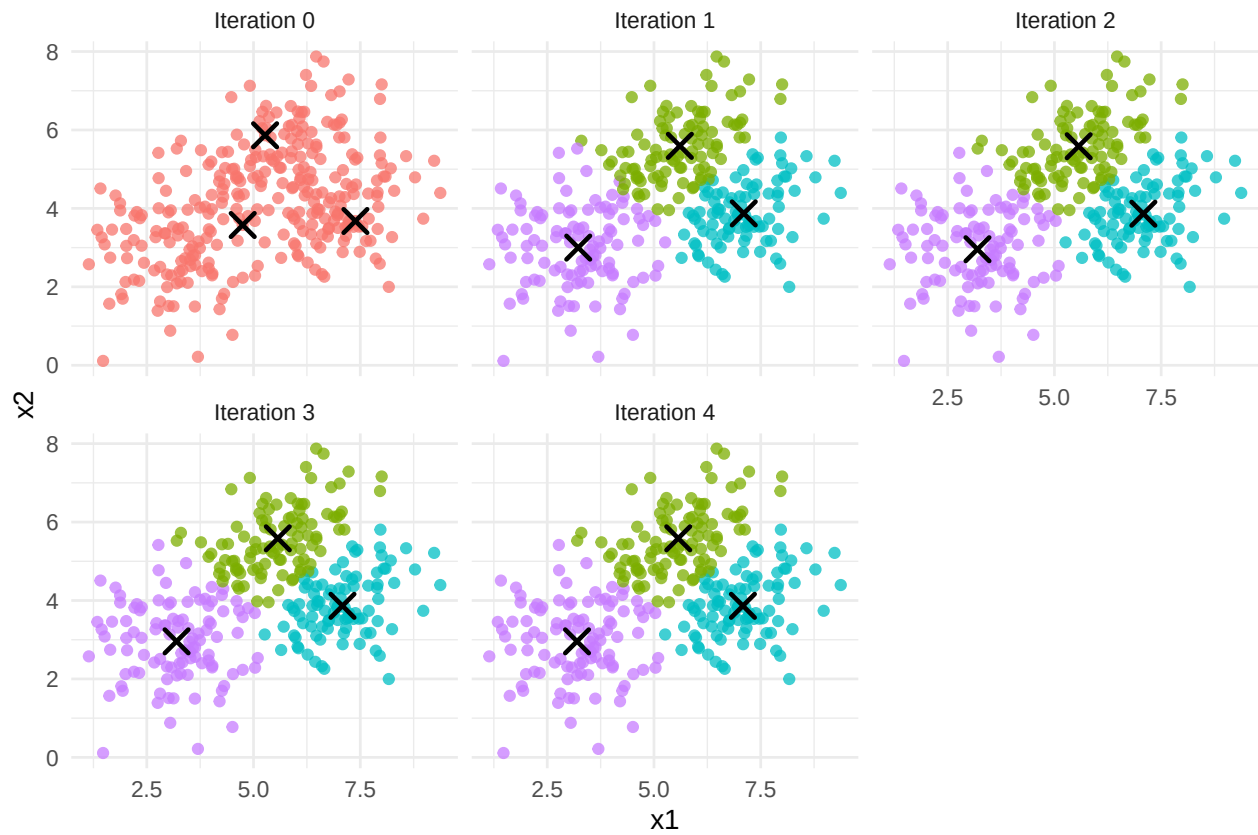
center_data[[iter + 1]] <- data.frame(
  x1 = centers[, 1],
  x2 = centers[, 2],
  iteration = paste("Iteration", iter)
)
}

iter_points <- do.call(rbind, iteration_data)
iter_centers <- do.call(rbind, center_data)

ggplot(iter_points, aes(x = x1, y = x2, color = cluster)) +
  geom_point(size = 1.6, alpha = 0.75) +
  geom_point(
    data = iter_centers,
    aes(x = x1, y = x2),
    inherit.aes = FALSE,
    size = 3.5,
    shape = 4,
    stroke = 1.3
  ) +
  facet_wrap(~ iteration) +
  labs(title = "K-means updates from one random initialization") +
  theme_minimal() +
  theme(legend.position = "none")

```

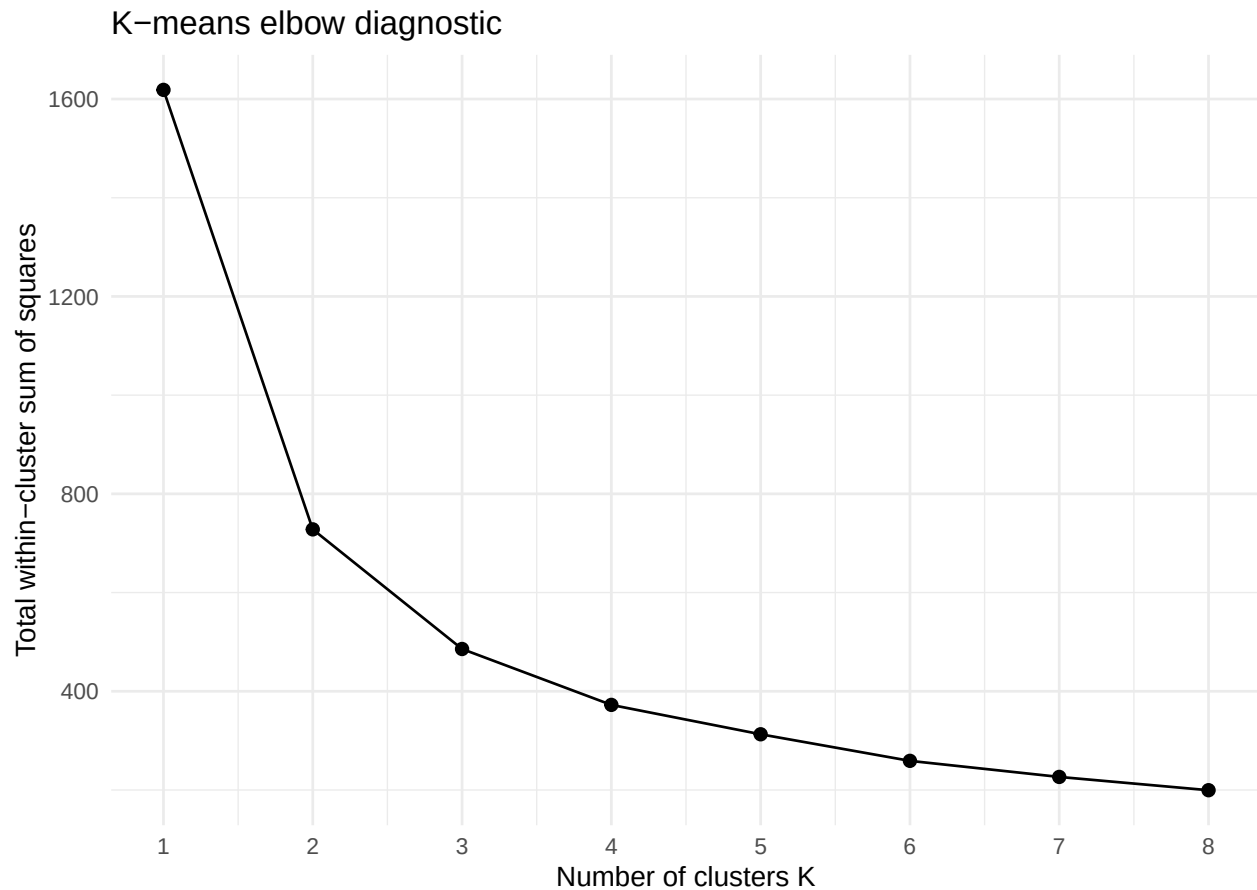
K-means updates from one random initialization



How should we choose K ?

The elbow plot is a diagnostic, not a theorem. In real data, the elbow is often ambiguous.

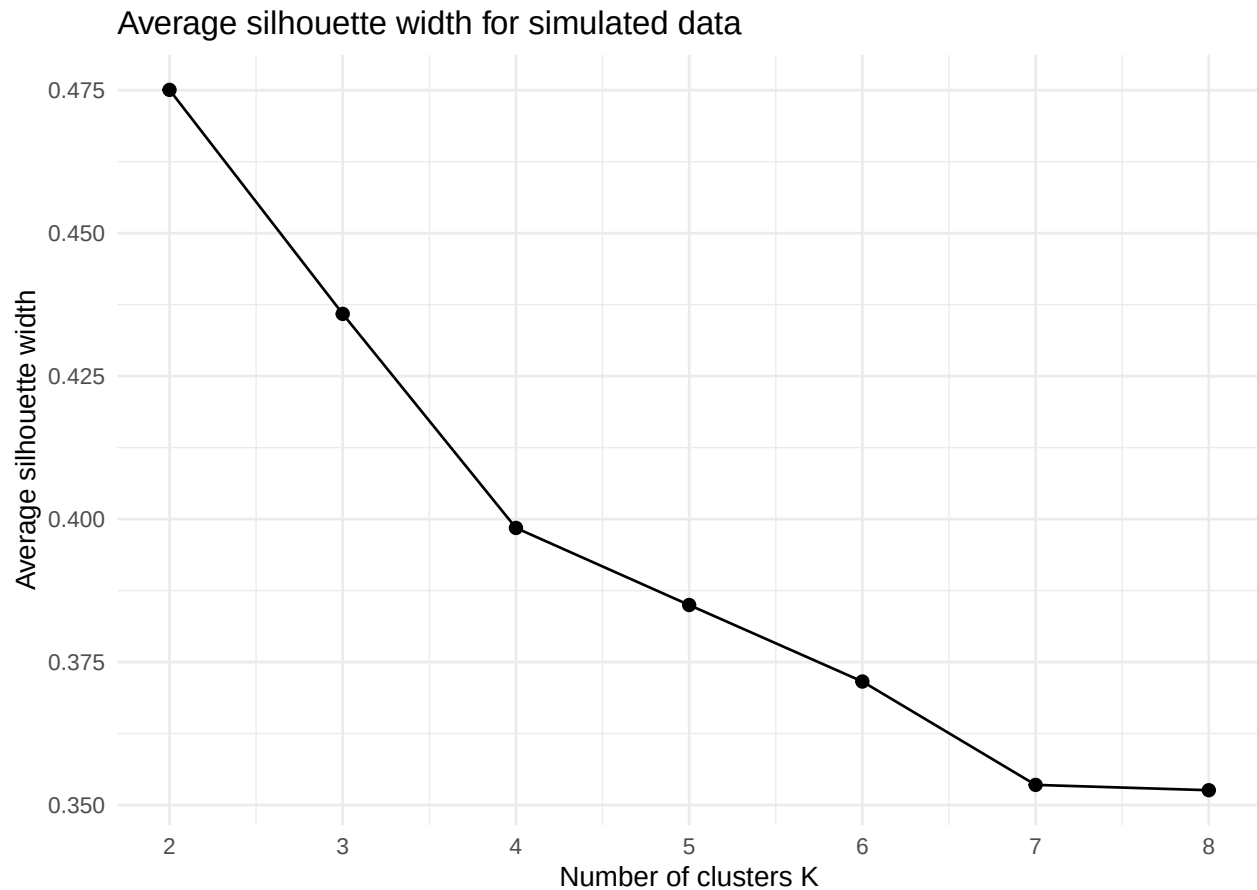
```
plot_kmeans_diagnostics(X_sim, k_grid = 1:8, nstart = 50)
```



If the `cluster` package is installed, we can also inspect average silhouette widths.

```
if (requireNamespace("cluster", quietly = TRUE)) {
  D_sim <- dist(X_sim)
  sil_data <- data.frame(
    K = 2:8,
    avg_silhouette = sapply(2:8, function(k) {
      set.seed(2107)
      cl <- kmeans(X_sim, centers = k, nstart = 50)$cluster
      mean(cluster::silhouette(cl, D_sim)[, "sil_width"])
    })
  )

  ggplot(sil_data, aes(x = K, y = avg_silhouette)) +
    geom_line() +
    geom_point(size = 2) +
    scale_x_continuous(breaks = 2:8) +
    labs(
      title = "Average silhouette width for simulated data",
      x = "Number of clusters K",
      y = "Average silhouette width"
    ) +
    theme_minimal()
} else {
  message("Install the cluster package to run the silhouette diagnostic.")
}
```



Data set 2: NCI60 microarray data

The NCI60 data contain expression measurements for 6,830 genes from 64 cancer cell lines. The cancer type is recorded, but clustering methods do not use those labels during fitting. We use the labels only to interpret the clusters after fitting.

```
NCI60 <- load_nci60()
ncidat_raw <- as.matrix(NCI60$data)
cancer_type <- as.factor(NCI60$labs)

ncidat_centered <- scale(ncidat_raw, center = TRUE, scale = FALSE)
ncidat_scaled <- scale(ncidat_raw, center = TRUE, scale = TRUE)

# Use informative row labels for plots. Duplicated cancer-type labels are allowed.
rownames(ncidat_raw) <- as.character(cancer_type)

dim(ncidat_raw)
```

```
## [1] 64 6830
```

```
table(cancer_type)
```

```
## cancer_type
##      BREAST      CNS      COLON K562A-repro K562B-repro  LEUKEMIA
##          7        5          7          1          1          6
## MCF7A-repro MCF7D-repro  MELANOMA      NSCLC      OVARIAN  PROSTATE
##          1          1          8          9          6          2
```

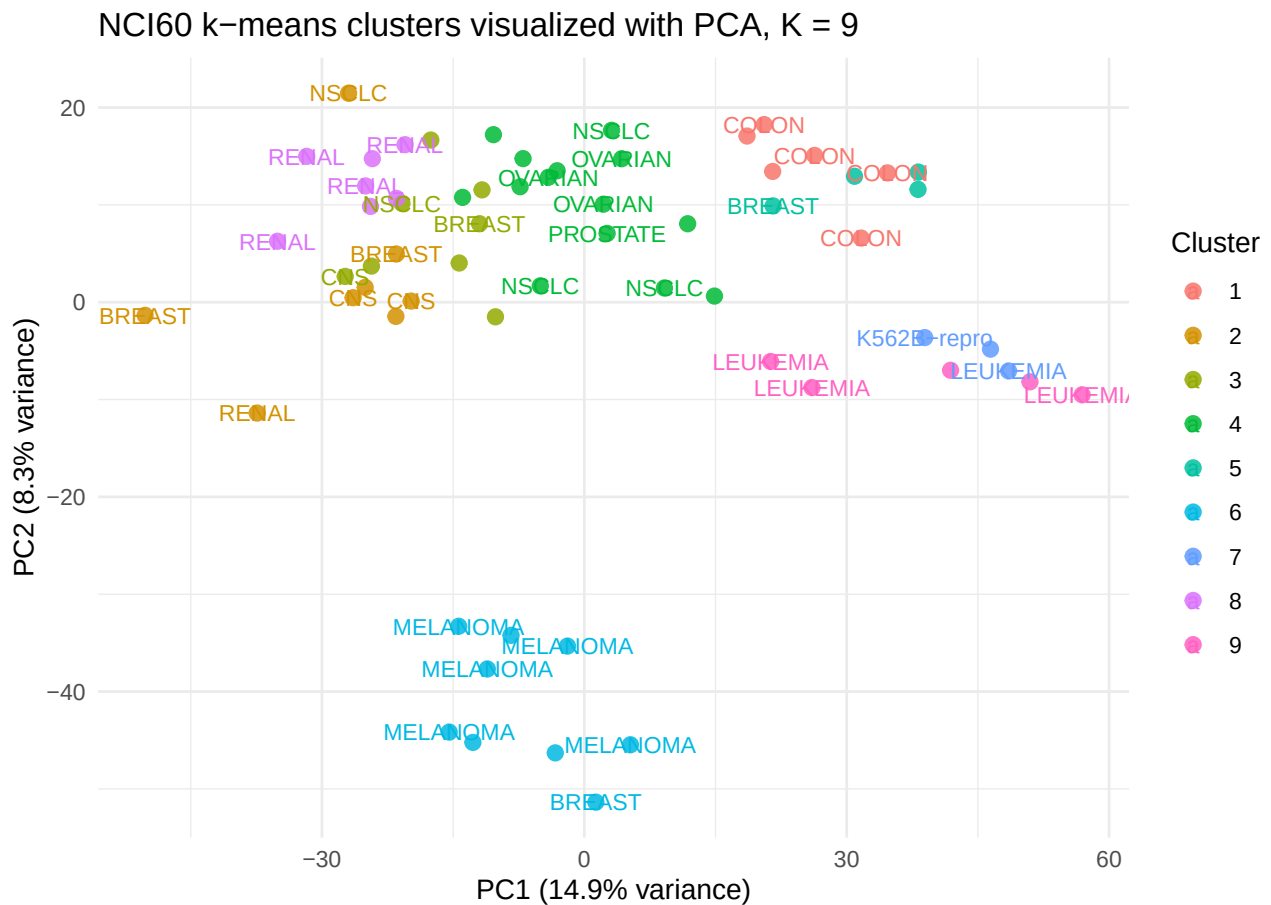
```
##      RENAL      UNKNOWN
##      9         1
```

K-means

```
set.seed(2107)
K_small <- 9
km_nci_k9 <- kmeans(ncidat_centered, centers = K_small, nstart = 5, iter.max = 100)

nci_scores_k9 <- pca_scores_for_plot(
  ncidat_centered,
  groups = km_nci_k9$cluster,
  labels = as.character(cancer_type),
  center = FALSE,
  scale = FALSE
)

plot_pca_clusters(
  nci_scores_k9,
  title = paste("NCI60 k-means clusters visualized with PCA, K =", K_small)
)
```

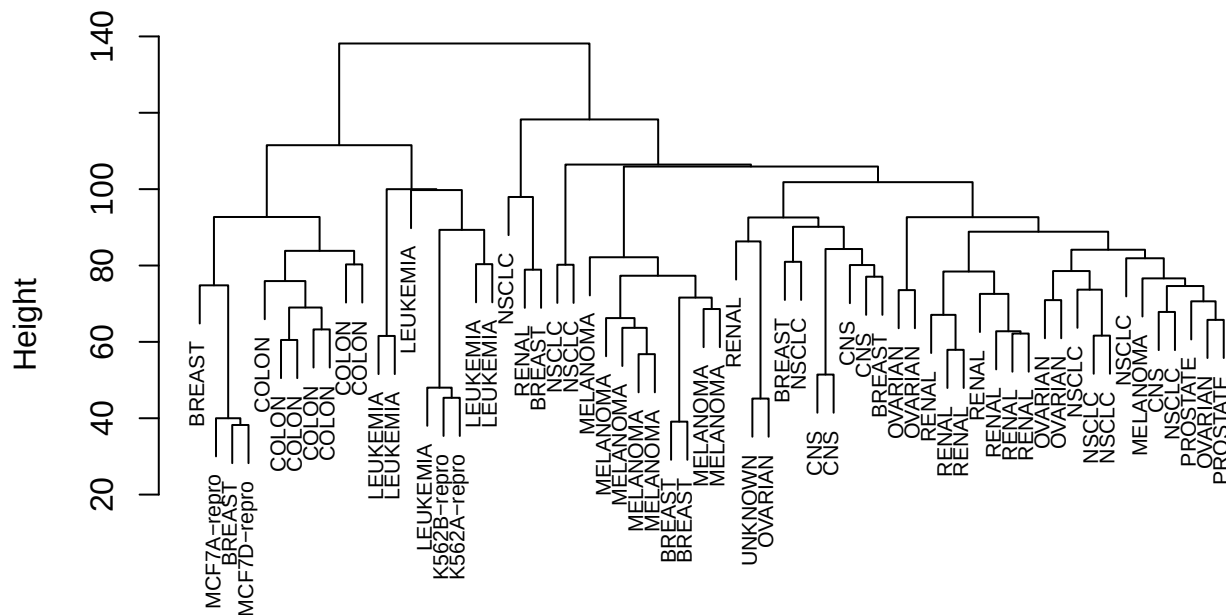


Hierarchical Clustering

Complete Linkage

```
D_euclidean <- dist(ncidat_centered, method = "euclidean")  
hc_complete <- hclust(D_euclidean, method = "complete")  
plot(hc_complete, labels = cancer_type, cex = 0.65, main = "Complete linkage")
```

Complete linkage



Cutting a Dendrogram

```
K_nci = 9  
hc_cut <- cutree(hc_complete, k = K_nci)  
table(Cluster = hc_cut, CancerType = cancer_type)
```

```
##      CancerType  
## Cluster BREAST  CNS  COLON K562A-repro K562B-repro LEUKEMIA MCF7A-repro  
##      1      2   4      0              0              0      0      0  
##      2      0   1      0              0              0      0      0  
##      3      1   0      0              0              0      0      0  
##      4      0   0      0              1              1      3      0  
##      5      0   0      0              0              0      2      0  
##      6      0   0      0              0              0      1      0
```

```
##      7      2      0      7      0      0      0      1
##      8      0      0      0      0      0      0      0
##      9      2      0      0      0      0      0      0
##      CancerType
## Cluster MCF7D-repro MELANOMA NSCLC OVARIAN PROSTATE RENAL UNKNOWN
##      1      0      0      1      1      0      1      1
##      2      0      1      5      5      2      7      0
##      3      0      0      1      0      0      1      0
##      4      0      0      0      0      0      0      0
##      5      0      0      0      0      0      0      0
##      6      0      0      0      0      0      0      0
##      7      1      0      0      0      0      0      0
##      8      0      0      2      0      0      0      0
##      9      0      7      0      0      0      0      0
```

```
adjusted_rand_index(cancer_type, hc_cut)
```

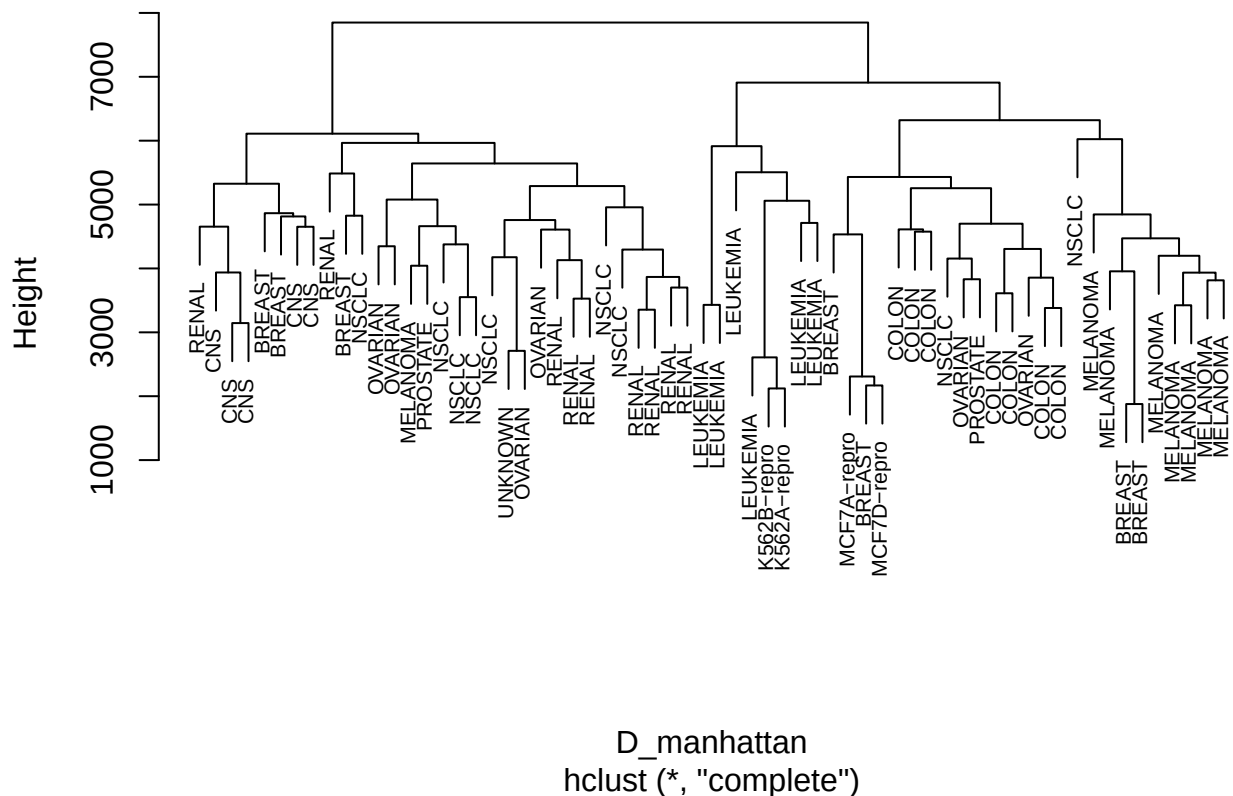
```
## [1] 0.2726685
```

Complete linkage with a different distance

```
D_manhattan <- dist(ncidat_centered, method = "manhattan")
hc_complete_l1 <- hclust(D_manhattan, method = "complete")

plot(hc_complete_l1, labels = cancer_type, cex = 0.65,
     main = "Complete linkage with Manhattan distance")
```

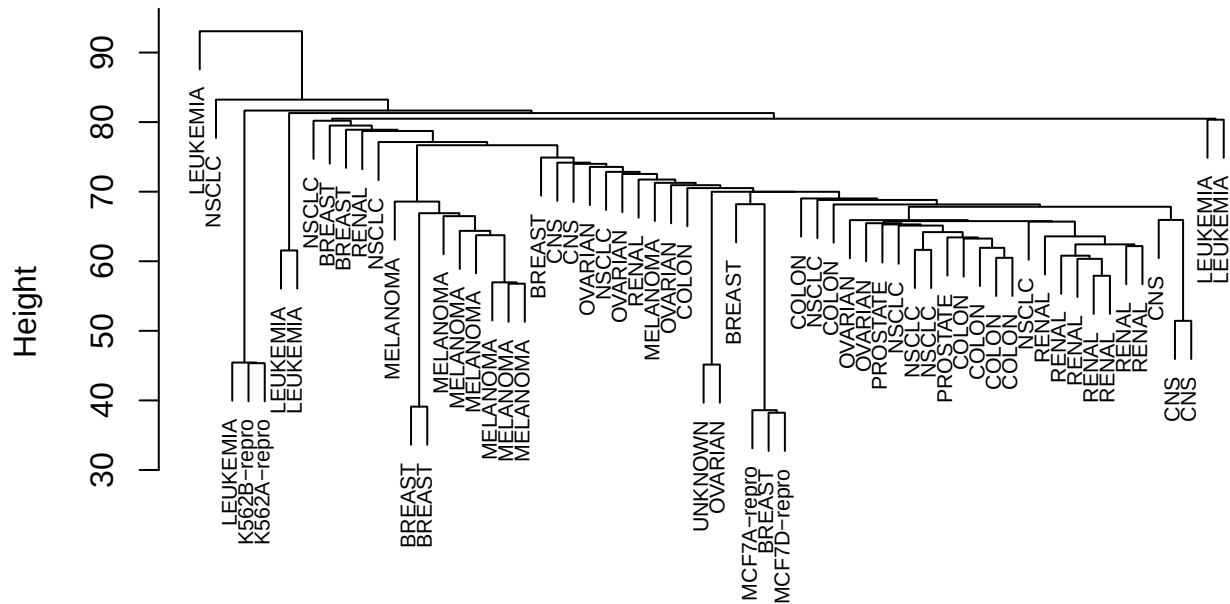
Complete linkage with Manhattan distance



Single Linkage

```
hc_single <- hclust(D_euclidean, method = "single")  
  
plot(hc_single, labels = cancer_type, cex = 0.65, main = "Single linkage")
```

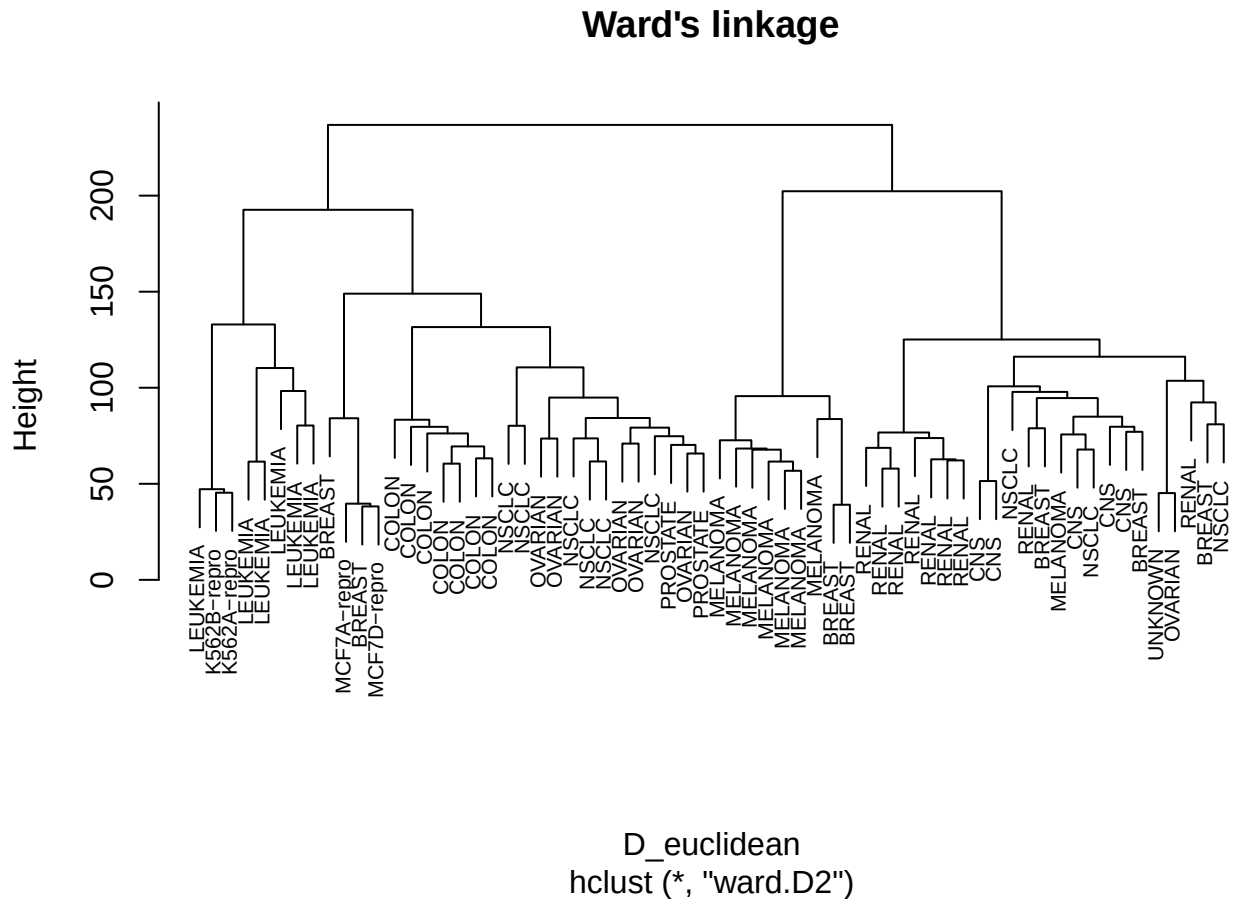
Single linkage



D_euclidean
hclust (*, "single")

Average Linkage

```
hc_average <- hclust(D_euclidean, method = "average")  
  
plot(hc_average, labels = cancer_type, cex = 0.65, main = "Average linkage")
```

Bicluster Heatmap

A full heatmap of all 6,830 genes is hard to read. One simple strategy is to use PCA loadings to select genes that strongly contribute to one of the leading PCs, then cluster both the samples and the selected genes.

```
pca_nci <- prcomp(ncidat_centered, center = FALSE, scale. = FALSE)

pc_to_use <- 2
n_genes <- 250
top_gene_index <- order(abs(pca_nci$rotation[, pc_to_use]), decreasing = TRUE)[1:n_genes]
heatmap_data <- ncidat_centered[, top_gene_index]

heatmap_data <- scale(heatmap_data, center = TRUE, scale = TRUE)
heatmap_data[is.na(heatmap_data)] <- 0

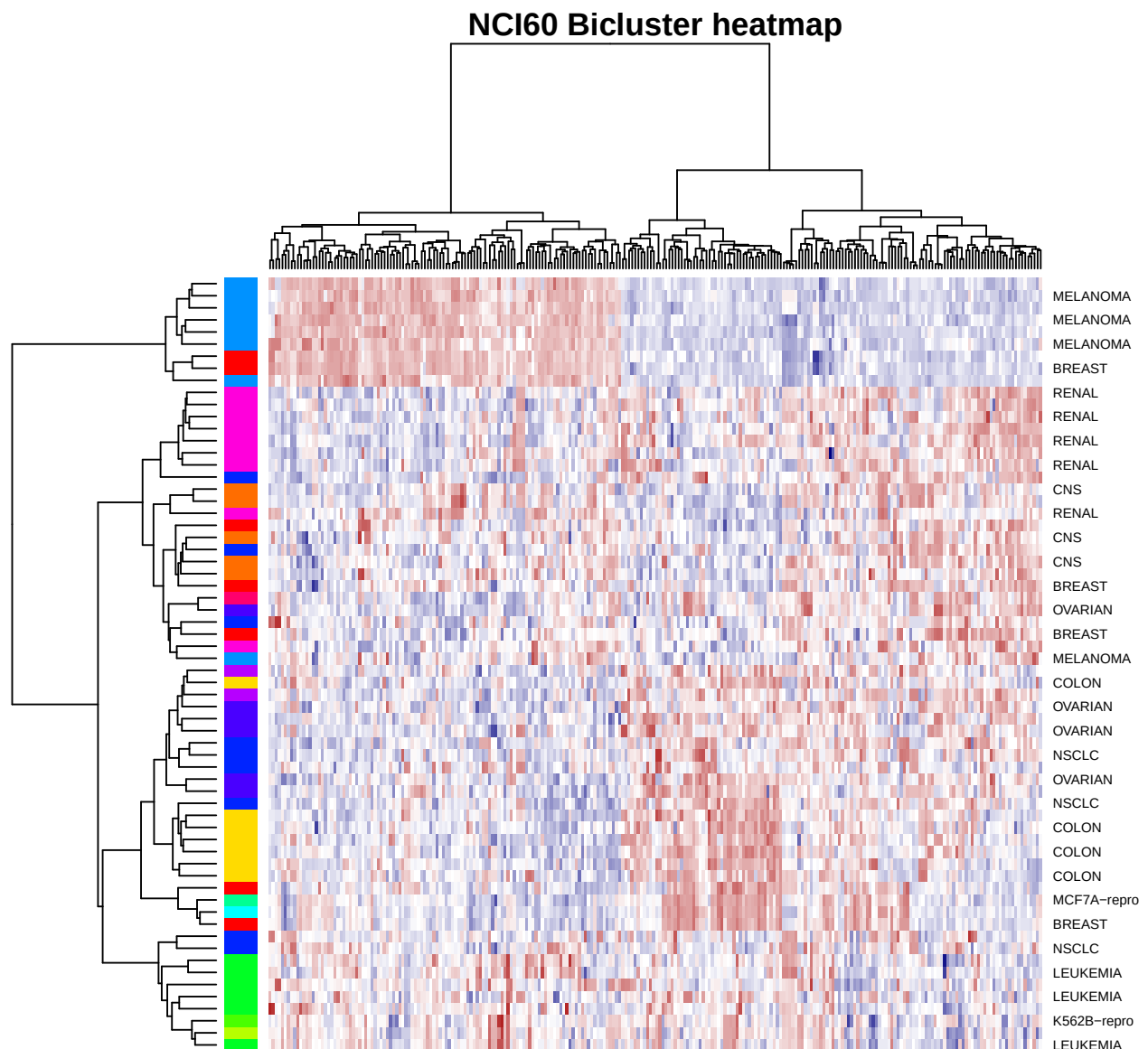
cancer_levels <- sort(unique(as.character(cancer_type)))
cancer_cols <- setNames(rainbow(length(cancer_levels)), cancer_levels)
row_side_cols <- cancer_cols[as.character(cancer_type)]

heatmap(
  heatmap_data,
  labRow = as.character(cancer_type),
  labCol = NA,
  RowSideColors = row_side_cols,
  col = colorRampPalette(c("navy", "white", "firebrick"))(75),
```

```

hclustfun = function(d) hclust(d, method = "ward.D2"),
main = paste("NCI60 Bicluster heatmap")
)

```



Spectral clustering

Toy example where k-means struggles

```

set.seed(2107)
n_circle <- 300
theta <- runif(n_circle, 0, 2 * pi)
inner <- cbind(cos(theta[1:(n_circle / 2)]), sin(theta[1:(n_circle / 2)])) +
  matrix(rnorm(n_circle, sd = 0.05), ncol = 2)
outer <- 2.3 * cbind(cos(theta[(n_circle / 2 + 1):n_circle]), sin(theta[(n_circle / 2 + 1):n_circle])) +
  matrix(rnorm(n_circle, sd = 0.10), ncol = 2)

```

```

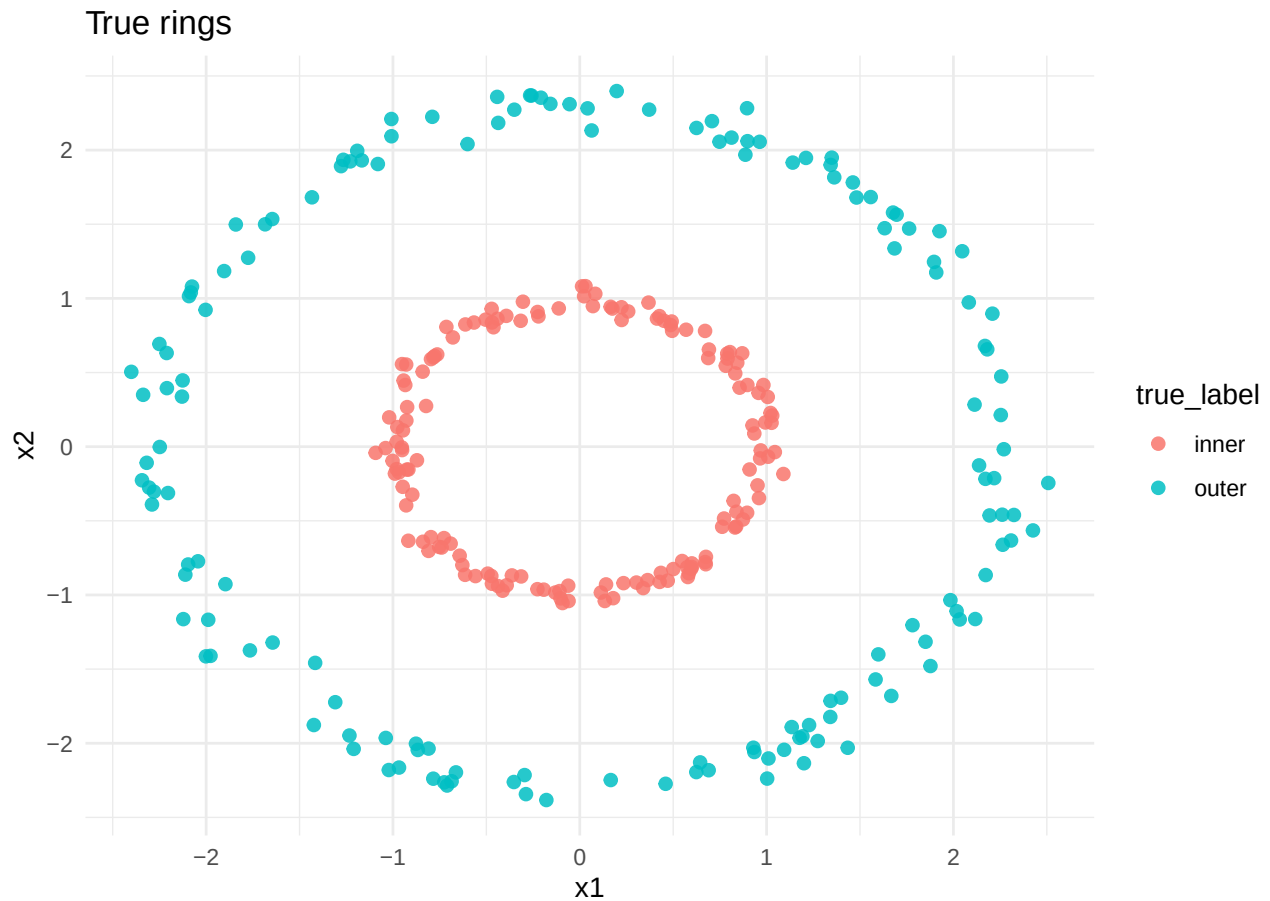
X_circle <- rbind(inner, outer)
true_circle <- factor(rep(c("inner", "outer"), each = n_circle / 2))
circle_data <- data.frame(x1 = X_circle[, 1], x2 = X_circle[, 2], true_label = true_circle)

set.seed(2107)
km_circle <- kmeans(X_circle, centers = 2, nstart = 50)
sc_circle <- spectral_clustering(X_circle, K = 2, n_neighbors = 10)

circle_data$kmeans <- factor(km_circle$cluster)
circle_data$spectral <- factor(sc_circle$cluster)

plot_cluster_solution(circle_data, cluster_var = "true_label", title = "True rings")

```

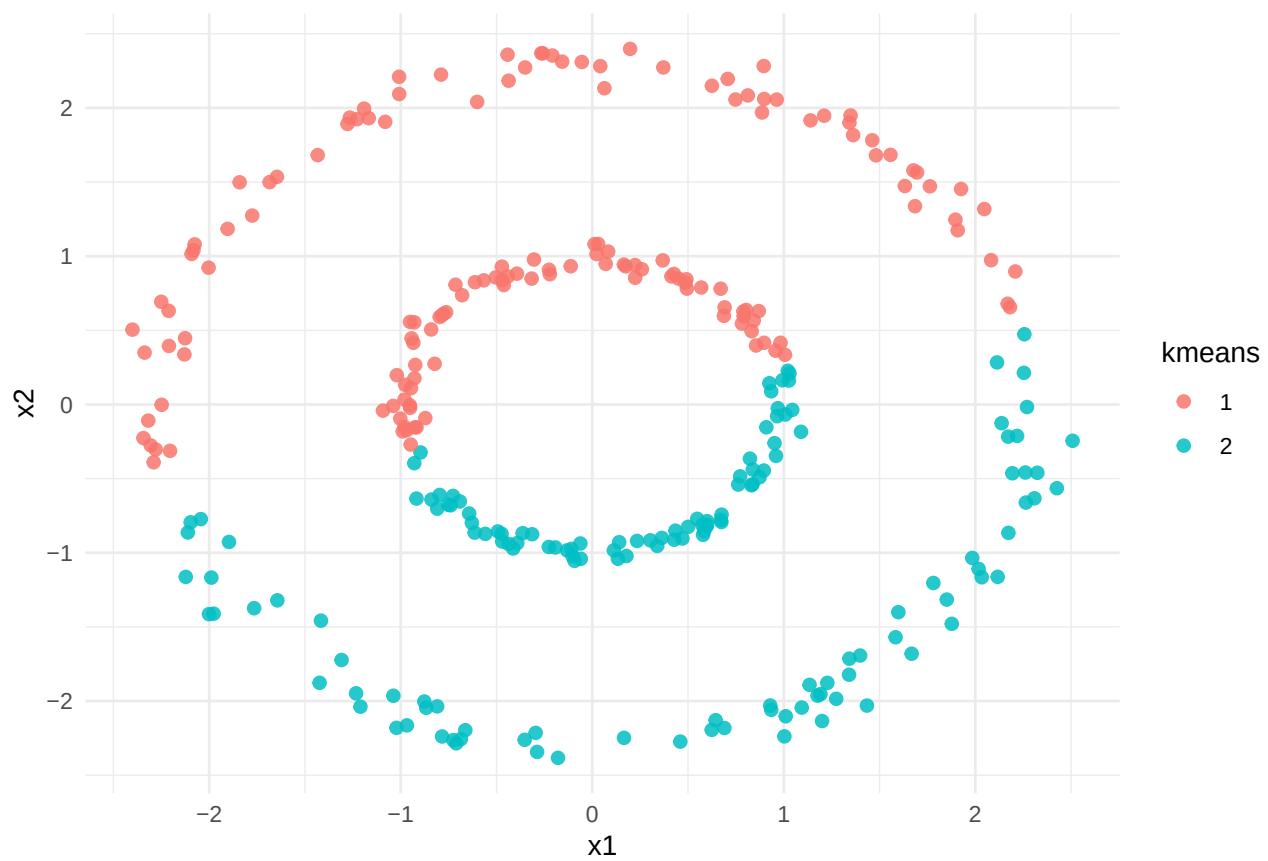


```

plot_cluster_solution(circle_data, cluster_var = "kmeans", title = "K-means on ring-shaped data")

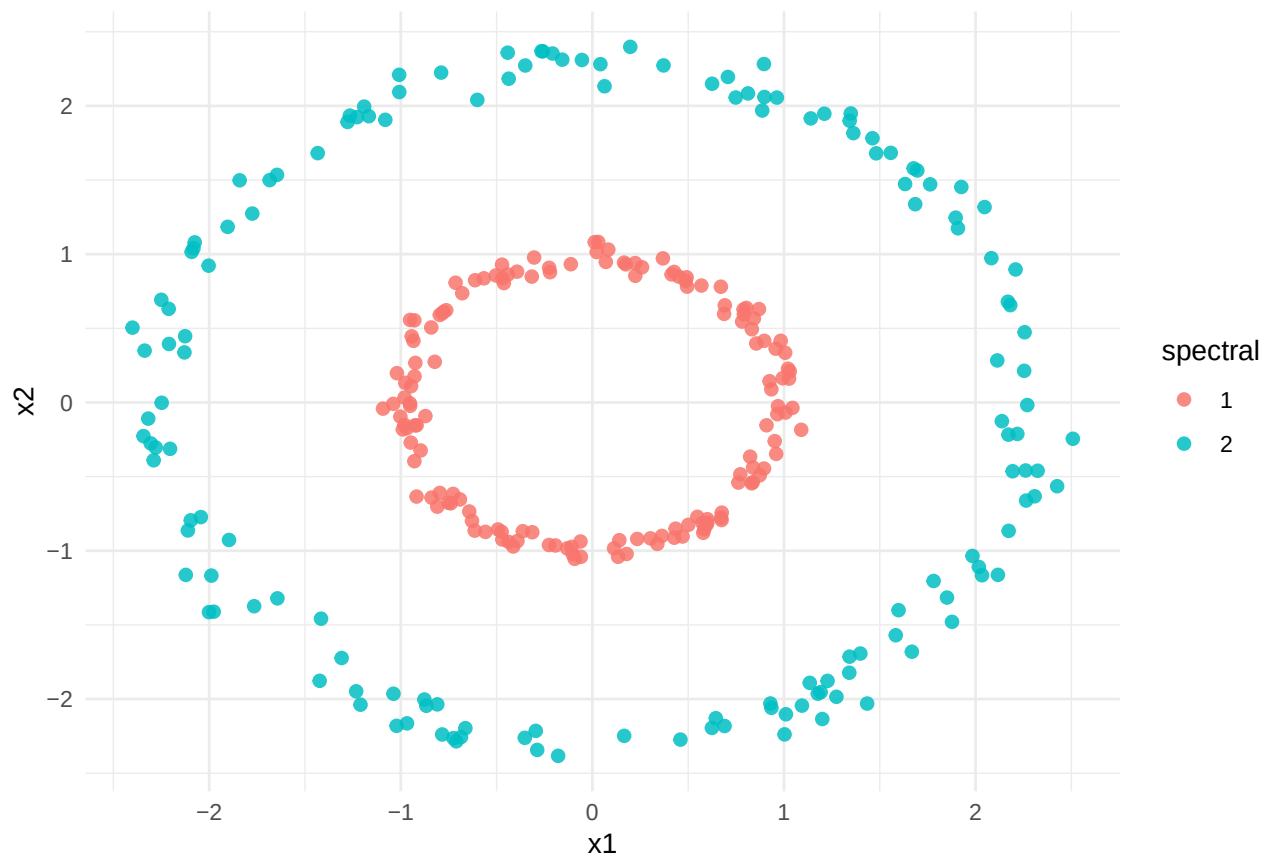
```

K-means on ring-shaped data



```
plot_cluster_solution(circle_data, cluster_var = "spectral", title = "Spectral clustering on ring-shaped data")
```

Spectral clustering on ring-shaped data



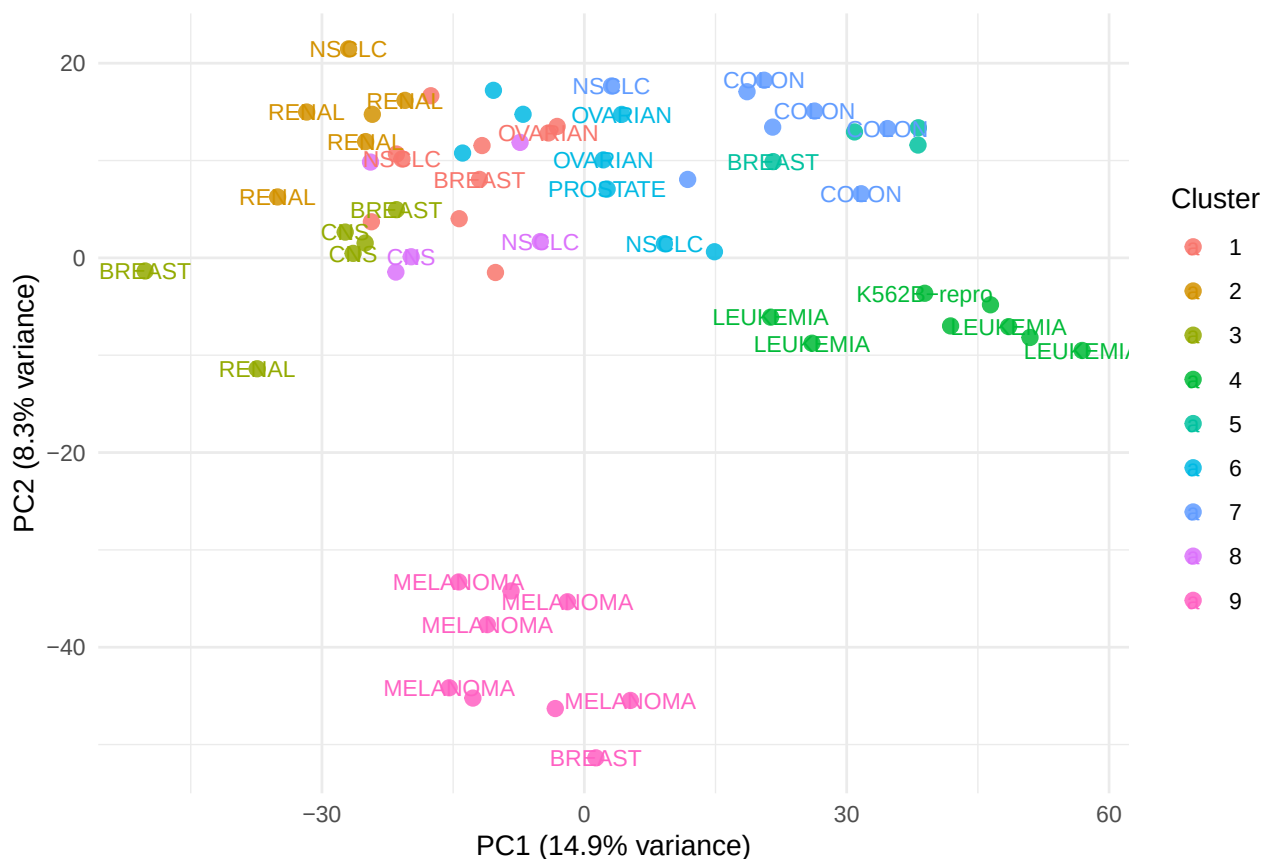
Spectral clustering on NCI60

```
set.seed(2107)
sc_nci <- spectral_clustering(ncidat_centered, K = K_nci, n_neighbors = 7)

nci_scores_sc <- pca_scores_for_plot(
  ncidat_centered,
  groups = sc_nci$cluster,
  labels = as.character(cancer_type),
  center = FALSE,
  scale. = FALSE
)

plot_pca_clusters(
  nci_scores_sc,
  title = paste("NCI60 spectral clustering visualized with PCA, K =", K_nci)
)
```

NCI60 spectral clustering visualized with PCA, K = 9



```
table(Cluster = sc_nci$cluster, CancerType = cancer_type)
```

```
##      CancerType
## Cluster BREAST  CNS  COLON K562A-repro K562B-repro LEUKEMIA MCF7A-repro
##      1      1  0    0          0          0          0          0
##      2      0  0    0          0          0          0          0
##      3      2  3    0          0          0          0          0
##      4      0  0    0          1          1          6          0
##      5      2  0    0          0          0          0          1
##      6      0  0    0          0          0          0          0
##      7      0  0    7          0          0          0          0
##      8      0  2    0          0          0          0          0
##      9      2  0    0          0          0          0          0
##      CancerType
## Cluster MCF7D-repro MELANOMA NSCLC  OVARIAN  PROSTATE  RENAL  UNKNOWN
##      1          0          1      2          2          1      1
##      2          0          0      1          0          0      5      0
##      3          0          0      0          0          0          1      0
##      4          0          0      0          0          0          0      0
##      5          1          0      0          0          0          0      0
##      6          0          0      3          4          1          0      0
##      7          0          0      1          0          0          0      0
##      8          0          0      2          0          0          1      0
##      9          0          7      0          0          0          0      0
```

```
adjusted_rand_index(cancer_type, sc_nci$cluster)
```

```
## [1] 0.3814899
```